

实现一个可以端到端交付全栈项目的“超级个体”

一、课题名称

实现一个可以端到端交付全栈项目的“超级个体”

二、课题要求（简介）

2.1 基础要求（MVP，所有完赛队伍最低交付）

本课题的最小功能闭环（MVP）如下：

1. **端到端跑通至少 1 条需求链路**：从 PM 自然语言输入 → 澄清 Agent → 方案 → 模块定位 → 代码生成 → 写入 Conduit 实仓 → Lint / 单测通过率 → 提交 PR
1. **真实接入 Conduit 仓库**：sandbox-repo 必须基于 **Conduit 仓库**（可 fork 后裁剪），**禁止使用与 Conduit 无关的 mock 仓库**；允许在 P0 阶段先在裁剪子集上跑通，P1 起在完整 Conduit 上稳定落地
2. **三端齐备**：前端对话页 + Node 后端 + AI 编排（Skill / Agent / Orchestrator）三层均有真实实现，不得仅做单端 demo
3. **可现场演示**：系统需能在合理时间内完成 L1 级别需求并通过 Lint + 单测
4. **所需资源**：使用项目组下发的 doubao-seed-2.0 lite EP / API key（见 § 八）；如需额外模型须自付费并在 README 声明

2.2 评判亮点（加分项，鼓励超越预期）

评判亮点用于在 MVP 之上拉开档位。本课题鼓励以下方向（命中越多档位越高，详细对照见 § 七）：

1. **抽象到位**：新增需求模式仅需新增 1 个 Skill 文件，不改主干
2. **断点重放**：流程任一阶段可暂停 / 修改 / 重放下游，事件溯源持久化
3. **跨栈一致性**：后端字段变更自动驱动前端类型 / Mock / 调用点更新
4. **可观测性**：每次 AI 调用记录 tokens / 延迟 / 成本，附简易监控面板
5. **业务上下文反哺**：历史需求结构化沉淀，相似新需求自动召回旧方案
6. **澄清深度**：能识别 PM 模糊需求中的歧义并主动追问，而非硬编生成

基于 AI CLI 工具、Agent SDK、开源或火山方舟模型等技术，以开源全栈博客仓库 **Conduit**

（<https://github.com/TonyMckes/conduit-realworld-example-app>，前后端单仓 monorepo）作为

实验田，实现一个可以端到端交付全栈项目的“超级个体”，要求能让产品经理在平台内通过对话完成“需求澄清 → 方案拆解 → 模块定位 → 代码生成 → 自动化测试 → 提测”全链路闭环；各阶段均可人工介入（追加澄清、修订方案、调整模块边界、修订测试用例等）；同时需设计一套可扩展的 **Skill / Agent / 流程编排** 分层，使新需求模式、新页面类型、新规范约束能以“注册”方式接入，而非改动主干；最终在该开源仓库上稳定产出可提测的代码变更，并沉淀业务上下文反哺后续需求生成，达到“显著降低 PM→研发翻译成本、提升需求吞吐”的目的。

三、课题背景

- 业务侧**：选用开源博客项目 **Conduit** 作为实验田，仓库地址 <https://github.com/TonyMckes/conduit-realworld-example-app>（React 18 + Vite + Express 4 + Sequelize + PostgreSQL，纯 JavaScript，前后端单仓 monorepo，一条 `git clone` 即可跑起来）。选型原因：(1) Medium 风格的完整博客，具备文章、标签、评论、用户、关注、收藏等典型全栈骨架，产品形态成熟、易于理解；(2) 高频模式化功能点（新增字段 / 新增筛选 / 新增页面 / 新增交互）天然丰富，非常适合模拟 PM→研发 的真实翻译链路；(3) 前后端同仓、技术栈简单，无任何内部依赖；(4) 体量适中，既有真实工程复杂度，又不会陷入海量代码噪声。
- 工程侧**：仓库为单体全栈架构（前端 React + 后端 Node.js），模块边界清晰、体量适中，既能体现真实工程的上下文复杂度，又不会陷入基础设施泥潭，便于聚焦在 AI 编排与抽象层本身。
- AI 侧**：当前 AI 在细节代码层已足够好，真正的壁垒在业务方法论、上下文工程、流程编排、Skill 抽象——即如何指挥 AI，而非 AI 能写多少行。
- 训练营目标**：通过一个贴近真实工程复杂度、但技术栈受限可控的课题，考察全栈能力、工程抽象能力、AI 心智，沉淀一套可复制的 AI 需求交付范式。

四、核心挑战

本课题需覆盖 **AI 工程业务全栈链路**——数据 / 上下文 → 模型 & Agent 编排 → 后端服务 → 前端交互 → 业务闭环。下表为本课题的七大挑战维度：

挑战维度	核心问题
上下文工程	如何在一个真实仓库里精准切出刚好够用的上下文喂给模型，避免 token 浪费与幻觉
需求澄清	如何设计澄清 Agent，让 PM 的模糊描述被自动结构化为可执行的需求 DSL
抽象能力	如何把“新增字段 / 新增页面 / 新增筛选 / 新增交互”等高频业务模式抽象为可复用 Skill
流程编排	澄清→方案→编码→测试→提测，每一步都必须可被人类打断、修订、重放
可验证性	生成的代码必须有自动化测试兜底，而不是“看起来对”就提测
全栈协同	前端组件 / 请求 / 类型与服务端接口 / 数据库 Schema 的跨栈一致性、联调自动化

工程化落地	在仓库原有的构建、依赖、测试、CI 约束下，保证生成的代码可被真实提交流水线接受
-------	--

五、导师信息

具体导师名单与沟通方式以课题群公布为准。

5.1 导师工作量预期

- 1. 频次：每周 1 次轻量同步（≤30 分钟），关键节点（启动会、中期 review、答辩）参与即可
- 2. 边界：仅提供方向性指导和比赛提供的相关基础能力问题回答

六、评分标准（对齐《AI 全栈项目挑战》官方权重）

本课题作品评分严格按官方下发的六维权重表执行。

评分维度	权重	官方考察要点	本课题观测口径
技术深度与创新性	30%	算法/模型选择是否合理；技术方案是否有独特或前瞻性思考	Skill / Agent / Orchestrator 分层是否原创且自治；上下文召回策略是否超越 RAG 暴力切片；澄清 Agent 对模糊需求的拆解深度
技术实现与工程完整度	25%	数据→Agent→后端→前端完整链路；前后端协同；可用性、性能、稳定性、可观测性；业务闭环	系统能在 Conduit 实仓上稳定生成可提测代码；Lint / 单测 / 集成测试绿；具备 token、时延、成本监控；至少 1 条端到端链路现场可演示
业务价值与场景契合度	20%	场景贴近企业级流程；可落地、可扩展；产品形态支撑业务价值；用户体验流畅	PM 视角的对话流是否自然、错误反馈是否友好、操作路径是否符合直觉；是否能识别歧义并主动追问；新增需求模式接入成本（改文件数、生效时间）；是否沉淀业务上下文反哺后续需求
代码质量与文档	10%	代码风格规范、可读性高；项目文档与注释齐全清晰	系统自身代码通过 Lint；README / 架构图 / 关键模块注释齐全；提交记录可追溯
数据与合规性	5%	数据隐私与安全；工具/模型/数据使用合规；备案与授权	使用豆包 EP / API key 仅限本课题；不外泄、不用于非挑战开发；如调用第三方模型需在文档说明并提供合规证明
项目材料完整度	10%	方案文档、演示视频、代码库材料完整规范；阐述清晰、演示直观	按 S 八清单全量交付：在线 Demo / 3-8min 演示视频 / 公开仓库 +

6.1 命题人补充：扣分项（任一触发即降档）

1. 过度依赖 prompt 硬编码——新增需求模式必须改主干，则 Skill 抽象不及格
2. 无任何自动化测试兜底，仅凭“看起来对”判断结果
3. 关键能力 mock / 伪实现（尤其是模型调用、仓库写入），未在 Conduit 实仓真实跑通
4. 套壳 ChatGPT 式 UI——只做了一个对话窗口贴模型输出，未接入仓库与测试链路

6.2 命题人补充：加分锚点（用于卓越项目排序）

1. **抽象到位**：现场出一道新模式题，学员只新增 1 个 Skill 文件即可支持，不改主干
2. **断点重放**：流程任一阶段可暂停、修改、重放下游，事件溯源式持久化
3. **跨栈一致性**：后端接口字段变更能自动驱动前端类型 / Mock / 调用点更新
4. **可观测性**：每次 AI 调用记录 tokens / 延迟 / 成本，有简单监控面板
5. **业务上下文反哺**：历史需求结构化沉淀，相似新需求能被自动召回旧方案

七、AI 辅助工具的使用要求（对齐官方资源池）

本课题严格遵循《AI 全栈项目挑战》官方下发的工具与资源使用规范，并叠加本课题的工程化要求。

7.1 官方资源池（每支队伍统一发放）

具体Trae配置步骤参考：[📖 TRAE IDE模型配置操作指南](#)

OpenAI Chat Completions API参考官方文档：

<https://www.volcengine.com/docs/82379/1494384?lang=zh>

1. **主模型**：**doubao-seed-2.0 lite**（火山方舟），由校招组统一申请，挑战周期内提供 EP + API key

代码块

```
1 EP: ep-20260514110933-mzh58
2 APIKEY: ark-3a6a7711-6bc2-444d-9572-f9e1887a7aab-40a6c
```

测试curl:

代码块

```
1 curl https://ark.cn-beijing.volces.com/api/v3/chat/completions \
2   -H "Authorization: Bearer ark-3a6a7711-6bc2-444d-9572-f9e1887a7aab-40a6c" \
3   -H "Content-Type: application/json" \
```

```
4 -d '{"model":"ep-20260514110933-mzh58","messages":
  [{"role":"user","content":"你好，请用一句话介绍你自己"}]}'
```

2. 使用规则：

- EP / API key **仅限本课题挑战开发使用**，严禁外泄、严禁用于非挑战项目
- 项目组实时观测调用流量，发现异常将直接封禁应用 / IP
- 受资源限制建议**错峰调用**（建议避开北京时间 14:00–18:00 高峰）；如配额不够请联系课题 HR 申请额外报销，配额查询入口由 TA 同步

3. **IDE 推荐**：建议优先使用 **TRAE**（参考官方下发的 TRAE IDE 模型配置指南）；不强制，可使用 Claude Code / Cursor 等其他 AI CLI

7.2 本课题对 AI 使用的额外要求

- 1. **过程留痕**：必须提交 AI 使用记录——关键对话、Skill 定义、Prompt 版本迭代、Agent 调用日志，作为评审材料的一部分（对应 § 八提交清单）
- 2. **禁止原样提交**：AI 生成的代码必须经过自身的理解、校验与改造，禁止未审阅直接提交到 sandbox-repo
- 3. **考察重点**：评分关注对 AI 的指挥能力、抽象设计能力、上下文工程能力——不是 AI 写了多少行
- 4. **合规要求**：涉及内部业务数据时必须脱敏；遵守公司 AI 工具使用规范、数据隐私与安全规范
- 5. **多模型组合**：除官方豆包外，允许自付费调用其他模型（OpenAI / Anthropic / 开源模型）作为对照或补充，但必须在 README 中显式声明模型清单与用途
- 6. **工具痕迹保留**：建议保留 AI 工具的配置、版本、Prompt 策略文件，便于答辩复盘

八、时间节点与提交要求

8.1 时间节点（对齐官方挑战周期）

节点	DDL	产出 / 动作
挑战正式启动	5.20	业务团队组织线上启动会，公开发布课题完整要求，集中答疑
项目实战期	5.20 – 6.10（约 3 周）	完成开发；导师每周 1 次方向性指导
项目成果提交	6.10 截止	按 § 8.2 清单全量提交：Demo / 视频 / 仓库 / README / 架构图 / AI 使用说明 等

答辩与评审	6.11 – 6.19	评审小组（资深技术专家 + HR）现场答辩、横向排序，评出卓越 1 + 优秀 ≤10
Offer 发放与表彰	6.19 之后	校招组发放奖金 / 直通终面 / 证书；如条件具备组织线下结项表彰会

8.2 提交材料清单（必填项）

所有完赛队伍必须按以下清单提交材料；缺项将影响「项目材料完整度」维度评分（占比 10%）。

字段组	字段	填写说明
基础信息	项目名称 / 课题	统一保持本课题命名
基础信息	团队名称与成员名单	列出成员姓名、学校、专业、角色
基础信息	分工说明（可选）	每位成员负责的模块（前端 / 后端 / Agent / Skill / 上下文工程 / 测试 / 部署）
功能说明	端到端使用流程	用 5–8 句写清 PM 从输入需求到拿到可提测 PR 的完整流程
交付材料	在线 Demo 链接（本地也行）	可直接访问；如需登录请提供体验账号或录屏替代
交付材料	演示视频链接	3–8 分钟，展示核心场景、关键功能、亮点与结果
交付材料	源代码仓库链接	GitHub / GitLab 公开仓库；包含 AI 系统主仓 + Conduit fork 子仓
交付材料	README / 运行说明	项目简介、依赖环境、启动步骤、目录结构、配置说明、API key 配置位置
技术说明	系统架构图	展示前端 / 后端 / 模型层 / Skill 注册表 / Orchestrator / sandbox-repo 调用关系
技术说明	关键工程难点与解决方案	至少 2–3 个：上下文召回精度 / 跨栈类型一致性 / Skill 注册热加载 / 断点重放等
技术说明	核心技术栈	分别说明前端 / 后端 / 数据库 / 中间件 / 部署环境 / 云资源等
结果说明	项目亮点 / 创新点	≤3 条，对应 § 6.2 加分锚点中命中的项

选填加分项：Prompt 策略 / Agent 流程图、评测方案与样例结果、性能指标（token 消耗 / 调用成功率 / 召回命中率）、用户反馈、版本迭代记录等。

九、公开练手题（供日常迭代参考）

以下题目用于帮助大家熟悉 Conduit 仓库与需求模式，**不代表答辩日最终抽题范围**。

9.1 L1 · 入门（纯前端 / 单模块）

1. **文章列表加阅读量字段**：在首页文章卡片上增加阅读量 icon + 数字展示（前端假数据即可，不改后端）
2. **Popular Tags 侧边栏前 5 个打标**：为接口返回的前 5 个标签增加视觉标识（纯前端取前 5，不引入排序语义）
3. **个人主页新增 About Me Tab**：在 Profile 页面现有 My Articles / Favorited Articles 之外新增一个 About Me Tab，展示 `User.bio`
4. **文章详情页新增字数统计**：在文章正文下方显示“本文共 XXX 字，预计阅读 X 分钟”，前端基于 `Article.body` 计算

9.2 L2 · 进阶（跨栈一致性）

1. **文章加封面图字段**：Article 模型加 `coverImage` 字段，新建/编辑文章表单支持输入 URL，列表卡片和详情页展示封面图
2. **评论支持点赞（含幂等）**：Comment 增加 `likeCount`，并设计幂等点赞机制
3. **文章草稿功能**：Article 增加 `status` 枚举（draft/published），编辑器新增“保存草稿”，列表默认过滤 draft，个人主页增加 Drafts Tab
4. **文章最后编辑时间展示**：利用 `updatedAt` 在文章详情页展示“最后编辑于 X 小时前”，同时后端保证 update 会刷新该字段

9.3 L3 · 挑战（仅公开题型）

答辩日 L3 题目将重点考察以下能力方向：

1. **模糊需求的主动澄清能力**：面对表达不完整、边界不清晰的需求，能否主动追问而非直接硬编
2. **多模块协同拆解能力**：面对涉及前端、后端、数据模型联动的需求，能否先拆解再落地
3. **反模式识别能力**：面对自相矛盾或定义不充分的需求，能否识别风险并给出合理实现路径

建议大家在准备时，不只练“写代码”，还要练“问对问题、拆对问题、验证对结果”。